Data Quality Findings Report

BrightLearn Data Warehouse Build

Source file: BrightLearn_Raw_Data.csv (5,000 rows)

Author: Tshifhiwa Tshikosi

Date: July 2026

Purpose

This report documents every data quality issue discovered in the source flat file during dataset inspection and pipeline development. For each issue it records what the problem is, how many records are affected, and the decision made to resolve it, or, where a decision was deliberately deferred, why.

Summary Table

#	Issue	Column(s)	Records Affected	Resolution
1	Inconsistent date formats	transaction_date	388 of 5,000 (7.8%)	Parsed with a multi-format TRY_CONVERT chain
2	Blank customer email	customer_email	811 of 5,000 (16.2%)	Replaced with placeholder value
3	Fully anonymous transactions (walk-in sales)	all customer fields	253 of 5,000 (5.1%)	Collapsed into one placeholder customer record
4	Named customers with no email ever recorded	customer_email	5 customers	Merged into the walk-in placeholder (known limitation)
5	Blank product category	category	132 of 5,000 (2.6%), 24 SKUs	Backfilled from other rows sharing the same SKU
6	Inconsistent casing in payment method	payment_method	4,446 of 5,000 (88.9%)	Standardised to a fixed set of display values
7	Inconsistent name casing for the same customer	customer_first_name	2,603 rows, 21 emails	Deduplicated using email as the matching key
8	Discount values inconsistent with a percentage	transaction_discount	1,213 of 5,000 (24.3%)	Identified; not resolved, flagged for clarification
9	Negative transaction amounts	transaction_amount	435 of 5,000 (8.7%)	Identified; not resolved , flagged for clarification
10	Unparsed date sub-format (yyyy/mm/dd)	transaction_date	88 of 5,000 (1.8%)	Identified during testing

Detailed Findings

1. Inconsistent date formats

Column: transaction_date **Affected:** 388 rows (7.8%) — the remaining 4,612 rows (92.2%) were in yyyy-mm-dd format.

The source file mixes at least four distinct date formats:

Format	Example	Row count
yyyy-mm-dd	2024-06-12	4,612
dd/mm/yyyy	17/05/2024	96
dd-mm-yyyy	04-04-2024	101
dd Mon yyyy	02 Feb 2024	103
Unrecognised	2024/06/05	88

Decision: Rather than assume a single format, dates were parsed using a COALESCE(TRY_CONVERT(...)) chain trying all four known formats in sequence, so any row matching one of them converts correctly regardless of position in the file.

Limitation carried forward: during pipeline testing, 88 rows (1.8%) turned out to be in a fifth format (yyyy/mm/dd, slash-separated) that the four-format chain does not catch. These rows are silently excluded from dim_date and, in turn, from fact_sales, since TRY_CONVERT returns NULL rather than erroring on an unmatched format. This is a known gap in the current cleaning logic rather than a resolved issue — a fifth TRY_CONVERT style would close it.

2. Blank customer email

Column: customer_email **Affected:** 811 rows (16.2%)

A meaningful share of transactions have no email recorded for the customer, even where a name is present.

Decision: Blank emails were replaced with a fixed placeholder (unknown@brightlearn.co.za) during cleaning, since email is used as the natural key for customer deduplication and cannot be left null in the dimension table. This was treated as a data quality fix rather than a modelling choice, since email is the only reliable natural key available in the source data.

3. Fully anonymous transactions (walk-in sales)

Columns: all customer fields **Affected:** 253 rows (5.1%)

253 transactions have no customer information recorded at all. first name, last name, and email are all blank. These are most likely walk-in or cash sales where no customer details were captured at the point of sale.

Decision: These rows are not distinct customers and should not be treated as 253 separate people. They were collapsed into a single dedicated "walk-in" customer record in dim_customer, sharing the same placeholder

email used for issue #2. This keeps the customer dimension meaningful,,l one row = one identifiable customer, or one deliberate anonymous bucket, rather than allowing an artificial customer count to build up on every load.

4. Named customers with no email ever recorded

Column: customer_email **Affected:** 5 customers

Cross-checking names against emails revealed 5 customers (e.g. Melissa Swanepoel, Hendrik Barnard, Salma Fredericks, Nozipho Cele, Anele Mbatha) who appear multiple times in the source data with a real name, but never with an email, very one of their transaction rows has a blank customer_email.

Decision: Because the customer dimension deduplicates on email, these 5 named customers are indistinguishable from the 253 anonymous walk-in sales (issue #3) once their email is defaulted to the same placeholder, all 258 rows collapse into one "walk-in" record. This is a **known and accepted limitation**, documented here rather than silently resolved: it means a small number of real, identifiable customers are being undercounted as anonymous. A more complete fix would fall back to matching on (first_name, last_name, phone_number) when email is blank but a name is present, which was not implemented within the scope of this project.

5. Blank product category

Column: category **Affected:** 132 rows (2.6%), spanning 24 distinct SKUs

Some rows have a blank category value even though the same SKU has a populated category on other rows in the file, the information exists in the dataset, just not on every row for that product.

Decision: Blank categories were backfilled using MAX(NULLIF(category,'')) OVER (PARTITION BY sku), pulling the known category for that SKU from any other row where it was recorded. This was possible because category is a fixed attribute of the product (identified by SKU), not something expected to vary row-to-row.

6. Inconsistent casing in payment method

Column: payment_method **Affected:** 4,446 rows (88.9%)

The same four payment methods appear in two different casings throughout the file (e.g. Cash and CASH, Credit Card and CREDIT CARD), which would otherwise cause the dimension load to treat them as 8 distinct payment methods instead of 4.

Decision: Standardised to one consistent display value per method (Cash, Credit Card, Debit Card, EFT, Store Credit) using a CASE expression matched on the upper-cased, trimmed value.

7. Inconsistent name casing for the same customer

Column: customer_first_name (and related name fields) **Affected:** 2,603 rows, across 21 distinct email addresses

For 21 customers, the same email address appears across multiple transactions with inconsistent name casing (e.g. Musa Ntuli in one row, MUSA NTULI in another). A naive SELECT DISTINCT across all customer columns treated these as separate people, since the casing difference made each row look unique.

Decision: Customer deduplication was changed to key strictly on email (using ROW_NUMBER() OVER (PARTITION BY email ...) to pick one representative row), rather than deduplicating across the full set of customer columns. This ensures a customer is recognised as the same person regardless of formatting inconsistencies elsewhere in their record.

8. Discount values inconsistent with a percentage

Column: transaction_discount **Affected:** 1,213 rows (24.3%)

transaction_discount appears intended to represent a percentage discount, but 1,213 rows contain values of 150, 200, 300, and 500, well outside a valid 0-100% range. This suggests either the column represents a Rand amount rather than a percentage for at least some transactions, or there is a data entry/export error affecting roughly a quarter of the file.

Decision: This issue was identified but not resolved within the scope of this project. Correctly interpreting these values requires clarification on the source system's intended meaning for this column rather than a one-sided assumption made during cleaning. It is flagged here so any revenue or discount-impact analysis built on this column is read with appropriate caution.

9. Negative transaction amounts

Column: transaction_amount **Affected:** 435 rows (8.7%)

435 rows have a negative transaction_amount. These may represent legitimate returns/refunds, or may be a data export defect, the source file gives no explicit indicator (e.g. a transaction type flag) to distinguish the two.

Decision: Also identified but not resolved. Silently converting these to positive values, or excluding them, would risk misrepresenting genuine refund activity if that is what they represent. This is flagged as a question to confirm with the business/data owner before deciding whether to model returns as a distinct transaction type.

Reflection

Several of these issues (dates, casing, blanks) were straightforward to resolve deterministically. Two of them — the discount anomaly and negative amounts, were deliberately left unresolved rather than "fixed" with an unverified assumption, since guessing wrong on either would silently distort every revenue calculation built downstream. Documenting an issue as known but unresolved is treated here as a valid and honest outcome of the data quality review, not a gap in the work.